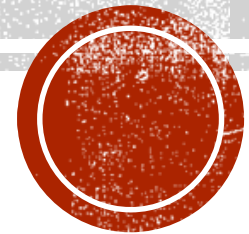


INTRODUCTION TO RECONFIGURABLE COMPUTING

CS G553: Reconfigurable computing

Dr. Meetha V Shenoy

Lect-9



LUT-based Technology Mapping

Algorithms to minimize area

Chortle-crf

MIS-fpga

Xmap

Algorithms delay minimization

FlowMap

*Chortle-
d*

*DAG-
map*

*MIS-fpga-
delay*

Algorithms for maximizing routability



FlowMap

Node Labelling

Labels every node in topological order

Each Node is processed after all its predecessors

Starting from PI nodes, compute node labels in topological order

Compute the label of a node based on labels of its predecessors

Node Mapping

Map the nodes to LUTs (K-Feasible cones) base on Labels



```

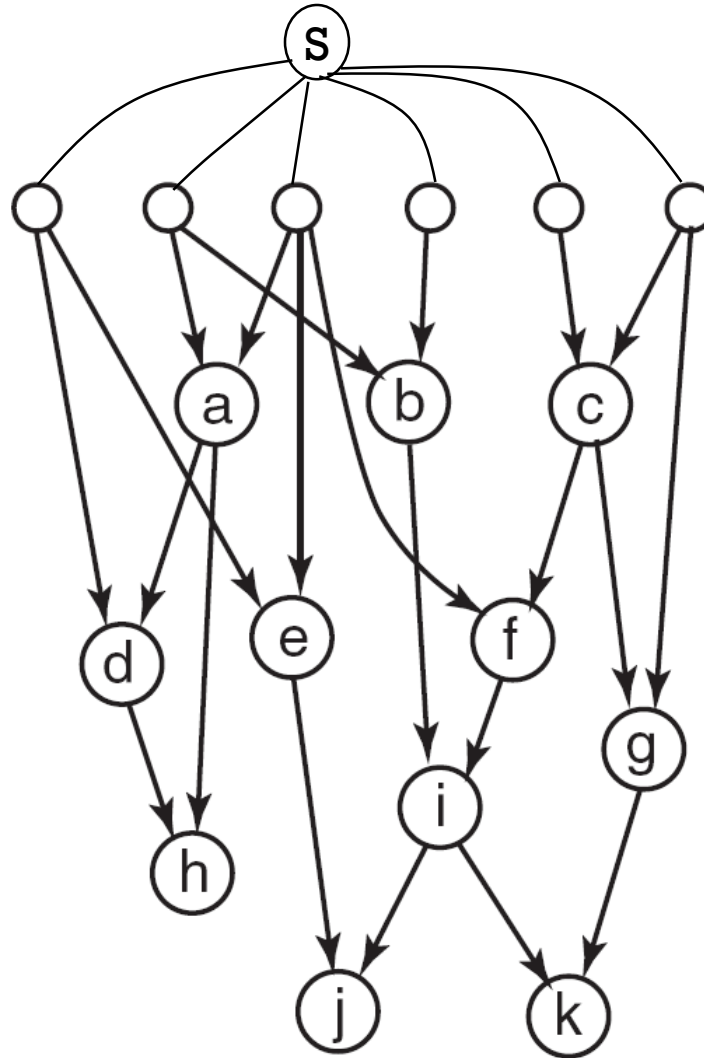
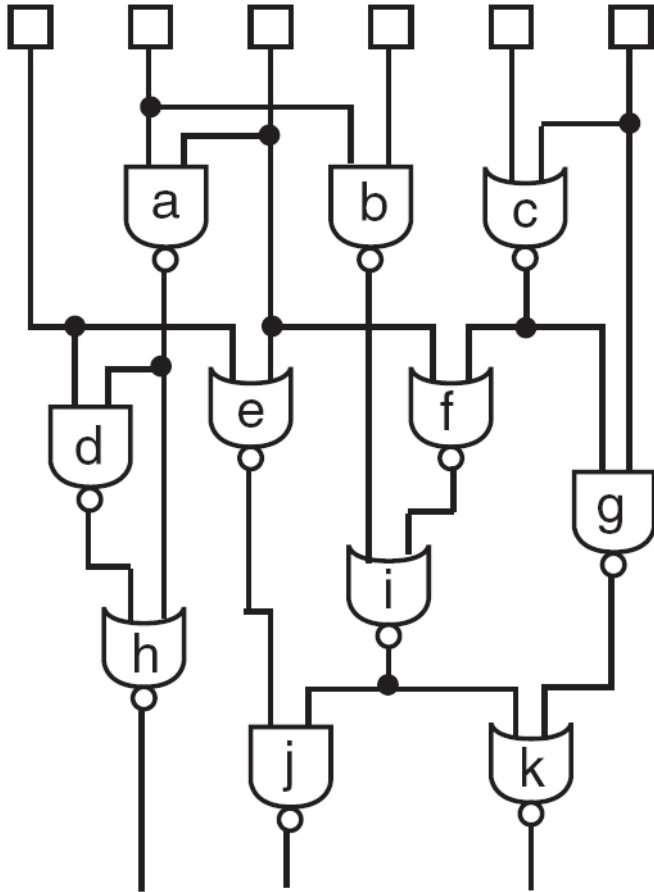
algorithm FlowMap
/* phase 1: labeling network */
for each PI node  $v$  do
     $l(v) := 0$ ;
 $T :=$  list of non-PI nodes in topological order;
while  $T$  is not empty do
    remove the first node  $t$  from  $T$ ;
    construct the network  $N_t$ ;
    let  $p = \max \{l(u) : u \in \text{input}(t)\}$ ;
    transform  $N_t$  into  $N'_t$  by collapsing all nodes in  $N_t$  with label  $p$  into  $t$ ;
    transform  $N'_t$  into  $N''_t$  as follows:
        split every node in  $\{x : x \in N'_t, x \neq s, x \neq t\}$  into two
        and connect them with a bridging edge of capacity 1;
        assign all non-bridging edges capacity  $\infty$ ;
    compute a cut  $(X'', \bar{X}'')$  in  $N''_t$  s.t.  $e(X'', \bar{X}'') \leq K$ 
        using the augmenting path algorithm;
    if  $(X'', \bar{X}'')$  is not found in  $N''_t$  then
         $\underline{X}_t := \{t\}$ ;  $l(t) := p + 1$ 
    else
        induce a cut  $(X, \bar{X})$  in  $N_t$  from the cut  $(X'', \bar{X}'')$  in  $N''_t$ ;
         $\underline{X}_t := \bar{X}$ ;  $l(t) := p$ 
    endif
endwhile;
/* phase 2: generate K-LUTs */

```



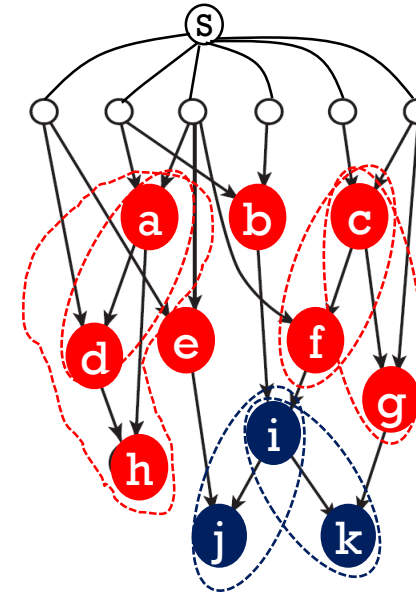
Node Labelling

Example: $K=3$



Topological order: {}

Node	Label	Clustering
a	1	$\{a\}$
b	1	$\{b\}$
c	1	$\{c\}$
d	1	$\{a, d\}$
e	1	$\{e\}$
f	1	$\{c, f\}$
g	1	$\{c, g\}$
h	1	$\{a, d, h\}$
i	2	$\{i\}$
j	2	$\{i, j\}$
k	2	$\{i, k\}$



```
/* phase 2: generate K-LUTs */  
 $L :=$  list of PO nodes;  
while  $L$  contains non-PI nodes do  
    take a non-PI node  $v$  from  $L$ ;  
    generate a K-LUT  $v'$  to implement the function of  $v$   
        such that  $input(v') = input(\overline{X_v})$ ;  
     $L := (L - \{v\}) \cup input(v')$   
endwhile  
end-algorithm;
```



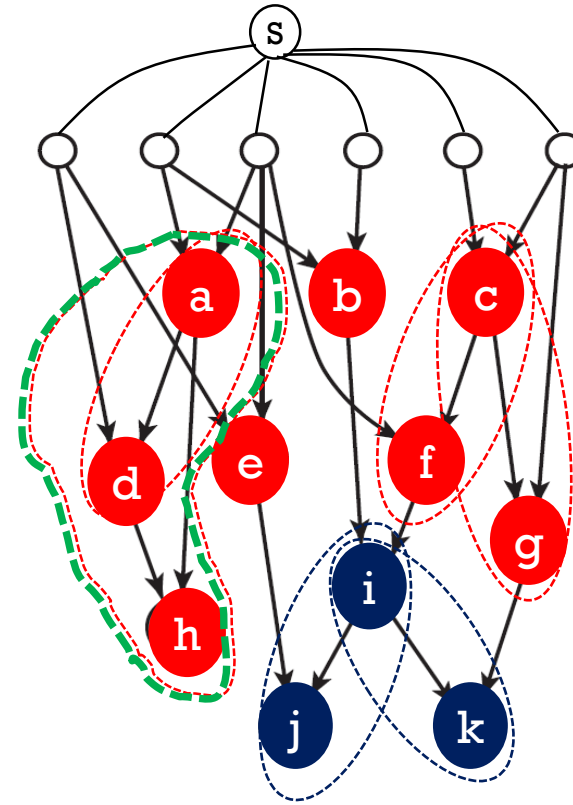
List of PO nodes $L = \{h, j, k\}$

$$\overline{X_h} = \{a, d, h\}$$

$\text{input}(\overline{X_h}) = \text{PI Nodes}$

$$L = (L - \{h\}) \cup \text{input}(\overline{X_h})$$

$$L = \{j, k, \text{PI nodes}\}$$



Node	Label	Clustering
<i>a</i>	1	{ <i>a</i> }
<i>b</i>	1	{ <i>b</i> }
<i>c</i>	1	{ <i>c</i> }
<i>d</i>	1	{ <i>a</i> , <i>d</i> }
<i>e</i>	1	{ <i>e</i> }
<i>f</i>	1	{ <i>c</i> , <i>f</i> }
<i>g</i>	1	{ <i>c</i> , <i>g</i> }
<i>h</i>	1	{ <i>a</i> , <i>d</i> , <i>h</i> }
<i>i</i>	2	{ <i>i</i> }
<i>j</i>	2	{ <i>i</i> , <i>j</i> }
<i>k</i>	2	{ <i>i</i> , <i>k</i> }



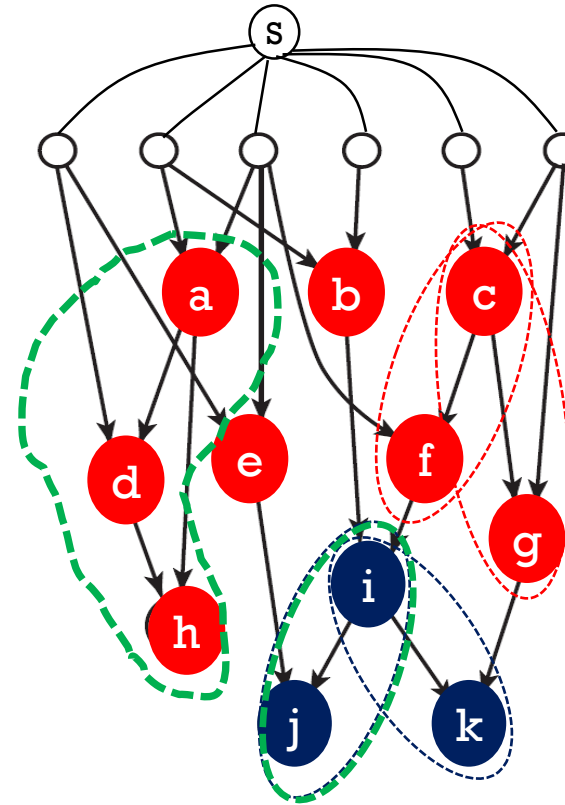
List of PO nodes $L = \{j, k, PI \text{ nodes}\}$

$$\bar{X}_j = \{i, j\}$$

$$\text{input}(\bar{X}_j) = \{e, f, b\}$$

$$L = (L - \{j\}) \cup \text{input}(\bar{X}_j)$$

$$L = \{k, PI \text{ nodes}, e, f, b\}$$



Node	Label	Clustering
a	1	$\{a\}$
b	1	$\{b\}$
c	1	$\{c\}$
d	1	$\{a, d\}$
e	1	$\{e\}$
f	1	$\{c, f\}$
g	1	$\{c, g\}$
h	1	$\{a, d, h\}$
i	2	$\{i\}$
j	2	$\{i, j\}$
k	2	$\{i, k\}$



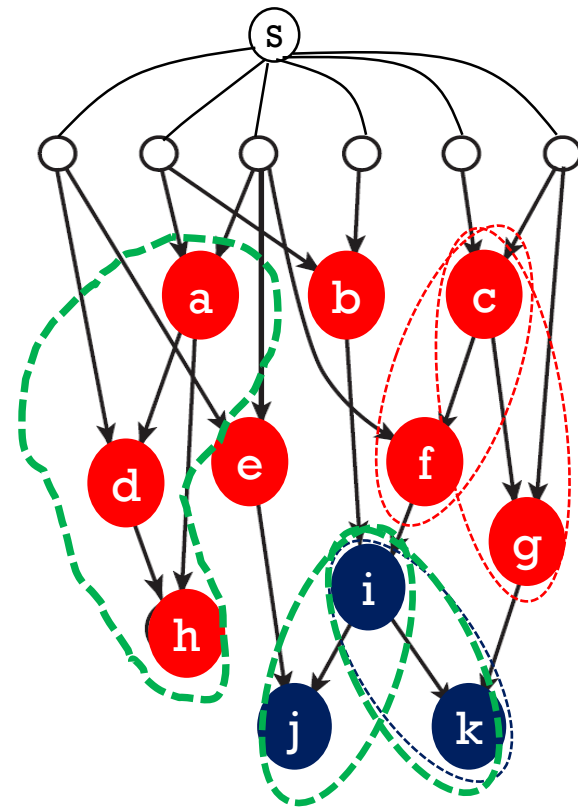
List of PO nodes $L = \{k, PI \text{ nodes}, e, f, b\}$

$$\overline{X_k} = \{i, k\}$$

$$input(\overline{X_k}) = \{b, f, g\}$$

$$L = (L - \{k\}) \cup input(\overline{X_k})$$

$$L = \{PI \text{ nodes}, e, b, f, g\}$$



Node	Label	Clustering
<i>a</i>	1	{ <i>a</i> }
<i>b</i>	1	{ <i>b</i> }
<i>c</i>	1	{ <i>c</i> }
<i>d</i>	1	{ <i>a</i> , <i>d</i> }
<i>e</i>	1	{ <i>e</i> }
<i>f</i>	1	{ <i>c</i> , <i>f</i> }
<i>g</i>	1	{ <i>c</i> , <i>g</i> }
<i>h</i>	1	{ <i>a</i> , <i>d</i> , <i>h</i> }
<i>i</i>	2	{ <i>i</i> }
<i>j</i>	2	{ <i>i</i> , <i>j</i> }
<i>k</i>	2	{ <i>i</i> , <i>k</i> }



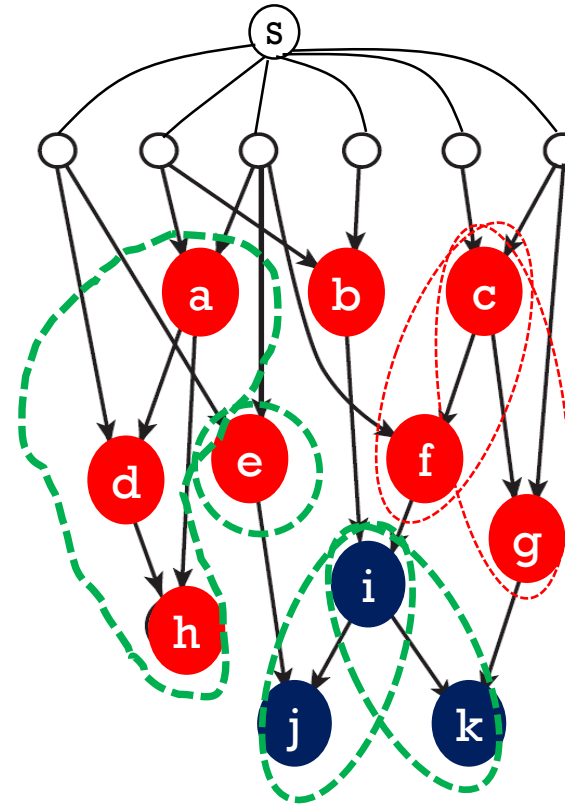
List of PO nodes $L = \{PI\ nodes, e, b, f, g\}$

$$\overline{X_e} = \{e\}$$

$$input(\overline{X_e}) = \{PI\ nodes\}$$

$$L = (L - \{e\}) \cup input(\overline{X_e})$$

$$L = \{PI\ nodes, b, f, g\}$$



Node	Label	Clustering
<i>a</i>	1	{ <i>a</i> }
<i>b</i>	1	{ <i>b</i> }
<i>c</i>	1	{ <i>c</i> }
<i>d</i>	1	{ <i>a</i> , <i>d</i> }
<i>e</i>	1	{ <i>e</i> }
<i>f</i>	1	{ <i>c</i> , <i>f</i> }
<i>g</i>	1	{ <i>c</i> , <i>g</i> }
<i>h</i>	1	{ <i>a</i> , <i>d</i> , <i>h</i> }
<i>i</i>	2	{ <i>i</i> }
<i>j</i>	2	{ <i>i</i> , <i>j</i> }
<i>k</i>	2	{ <i>i</i> , <i>k</i> }



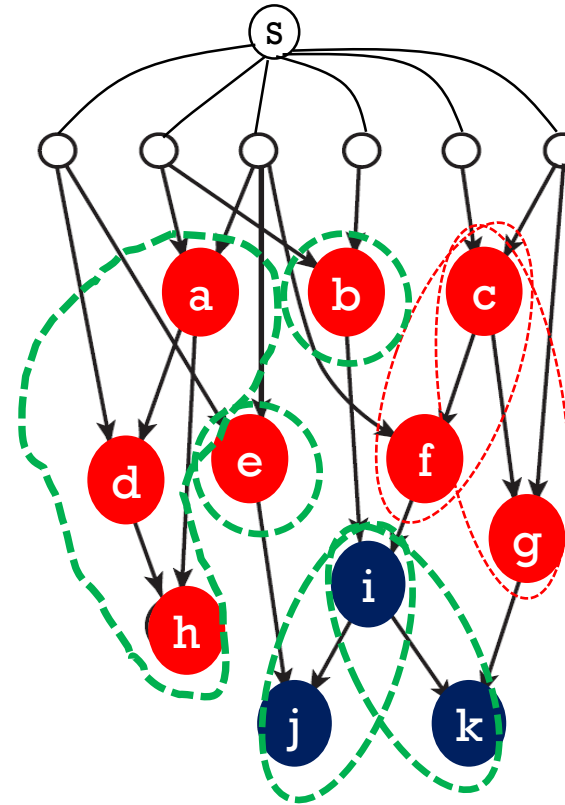
List of PO nodes $L = \{PI\ nodes, b, f, g\}$

$$\overline{X_b} = \{b\}$$

$$input(\overline{X_b}) = \{PI\ nodes\}$$

$$L = (L - \{b\}) \cup input(\overline{X_b})$$

$$L = \{PI\ nodes, f, g\}$$



Node	Label	Clustering
<i>a</i>	1	{ <i>a</i> }
<i>b</i>	1	{ <i>b</i> }
<i>c</i>	1	{ <i>c</i> }
<i>d</i>	1	{ <i>a, d</i> }
<i>e</i>	1	{ <i>e</i> }
<i>f</i>	1	{ <i>c, f</i> }
<i>g</i>	1	{ <i>c, g</i> }
<i>h</i>	1	{ <i>a, d, h</i> }
<i>i</i>	2	{ <i>i</i> }
<i>j</i>	2	{ <i>i, j</i> }
<i>k</i>	2	{ <i>i, k</i> }



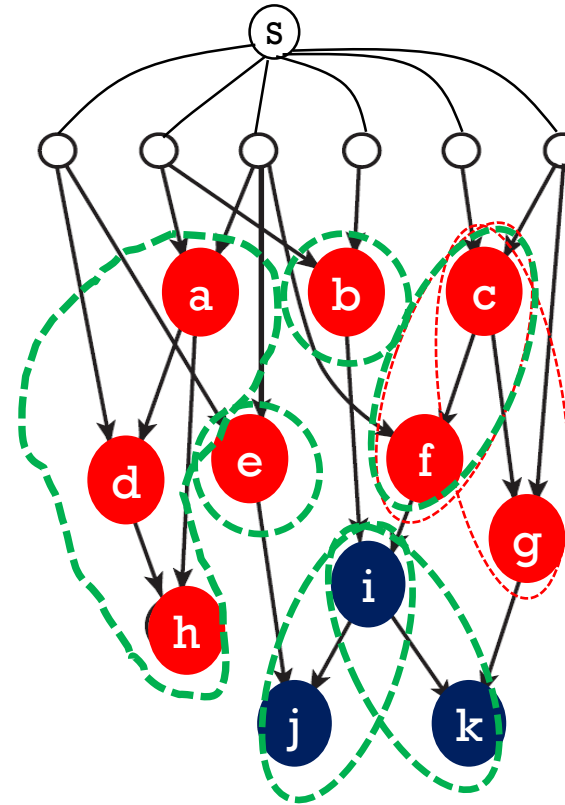
List of PO nodes $L = \{PI\ nodes, f, g\}$

$$\overline{X_f} = \{c, f\}$$

$$input(\overline{X_f}) = \{PI\ nodes\}$$

$$L = (L - \{f\}) \cup input(\overline{X_f})$$

$$L = \{PI\ nodes, g\}$$



Node	Label	Clustering
<i>a</i>	1	{ <i>a</i> }
<i>b</i>	1	{ <i>b</i> }
<i>c</i>	1	{ <i>c</i> }
<i>d</i>	1	{ <i>a</i> , <i>d</i> }
<i>e</i>	1	{ <i>e</i> }
<i>f</i>	1	{ <i>c</i> , <i>f</i> }
<i>g</i>	1	{ <i>c</i> , <i>g</i> }
<i>h</i>	1	{ <i>a</i> , <i>d</i> , <i>h</i> }
<i>i</i>	2	{ <i>i</i> }
<i>j</i>	2	{ <i>i</i> , <i>j</i> }
<i>k</i>	2	{ <i>i</i> , <i>k</i> }



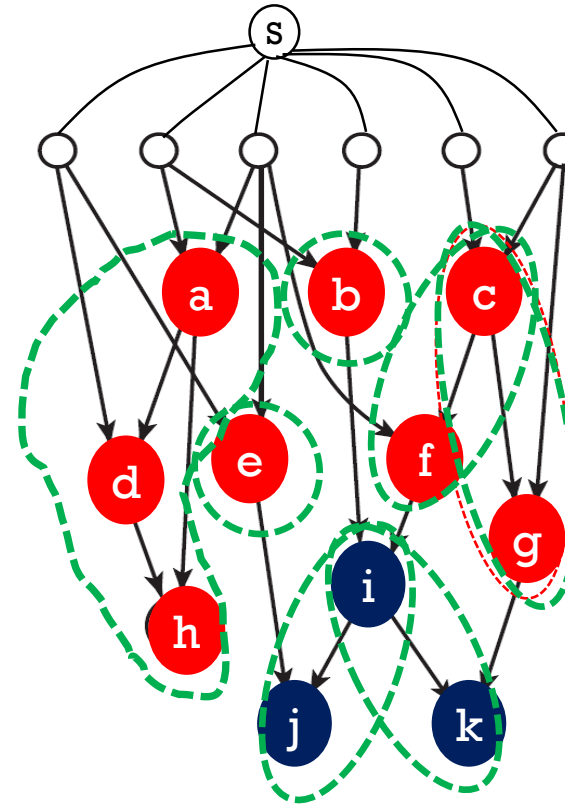
List of PO nodes $L = \{PI\ nodes, g\}$

$$\overline{X_g} = \{c, g\}$$

$$input(\overline{X_g}) = \{PI\ nodes\}$$

$$L = (L - \{g\}) \cup input(\overline{X_g})$$

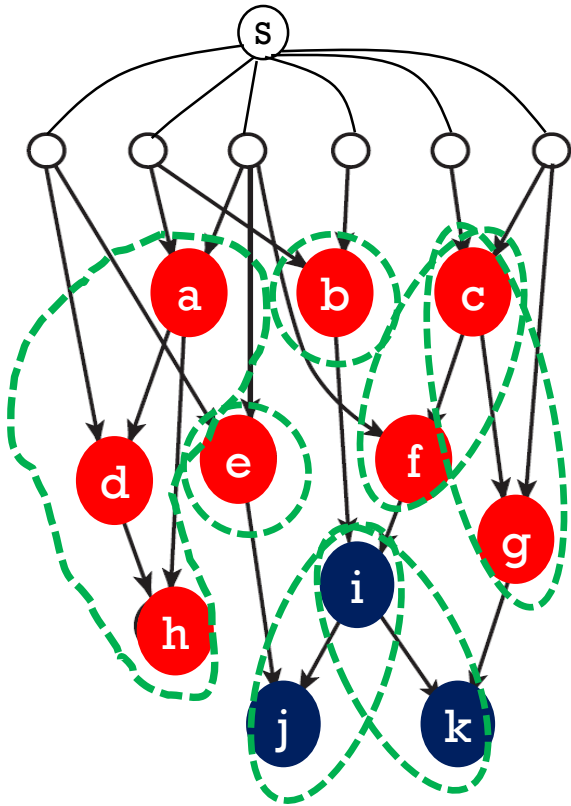
$$L = \{PI\ nodes\}$$



Node	Label	Clustering
<i>a</i>	1	{ <i>a</i> }
<i>b</i>	1	{ <i>b</i> }
<i>c</i>	1	{ <i>c</i> }
<i>d</i>	1	{ <i>a, d</i> }
<i>e</i>	1	{ <i>e</i> }
<i>f</i>	1	{ <i>c, f</i> }
<i>g</i>	1	{ <i>c, g</i> }
<i>h</i>	1	{ <i>a, d, h</i> }
<i>i</i>	2	{ <i>i</i> }
<i>j</i>	2	{ <i>i, j</i> }
<i>k</i>	2	{ <i>i, k</i> }



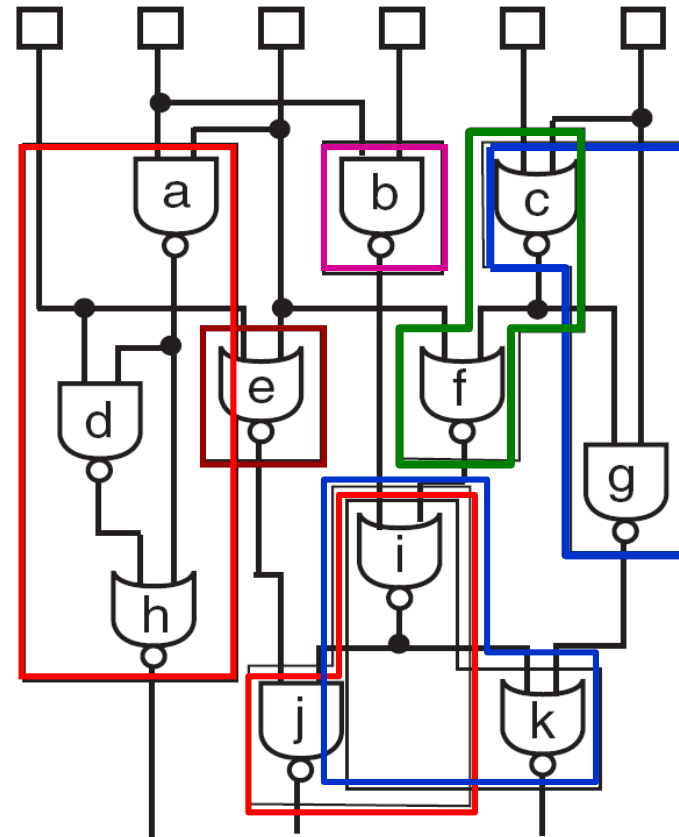
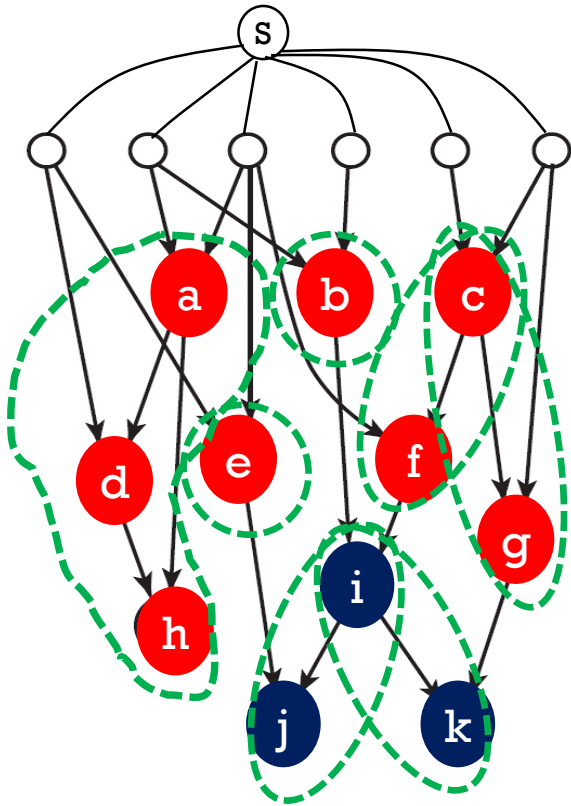
List of PO nodes $L = \{PI\ nodes\}$



Root	Elements
h	$\{a, d, h\}$
j	$\{i, j\}$
k	$\{i, k\}$
e	$\{e\}$
b	$\{b\}$
f	$\{c, f\}$
g	$\{c, g\}$



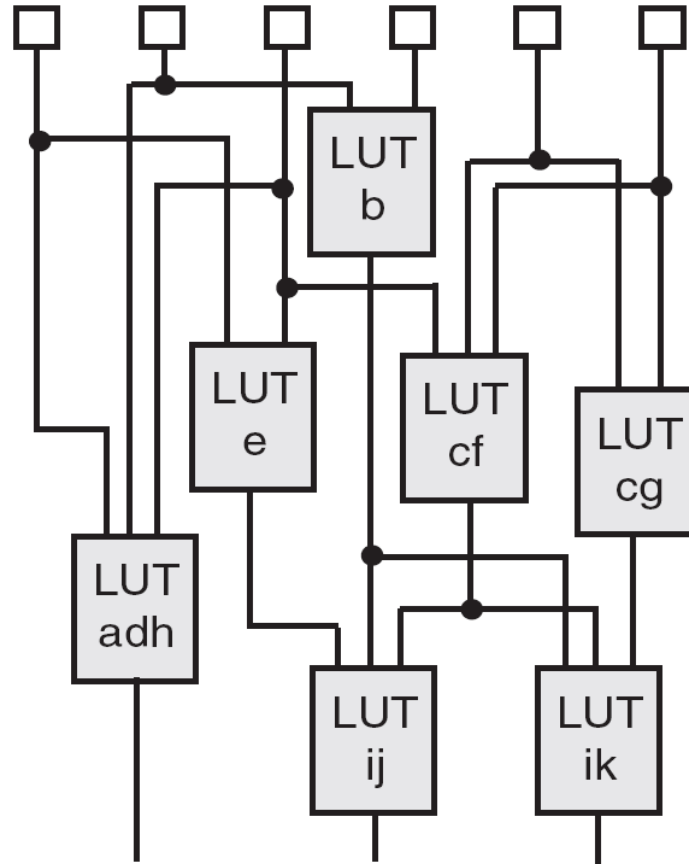
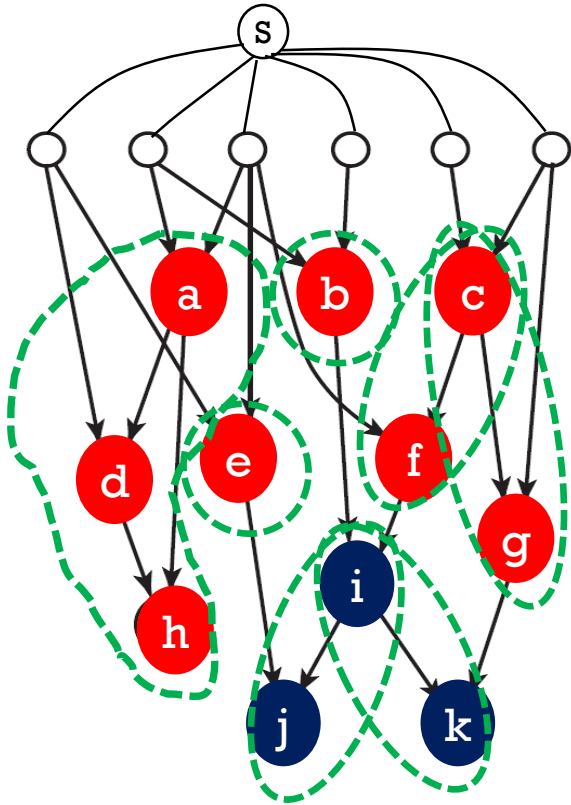
List of PO nodes $L = \{PI\ nodes\}$



Root	Elements
h	$\{a, d, h\}$
j	$\{i, j\}$
k	$\{i, k\}$
e	$\{e\}$
b	$\{b\}$
f	$\{c, f\}$
g	$\{c, g\}$



List of PO nodes $L = \{PI\ nodes\}$



```

/* phase 2: generate K-LUTs */
L := list of PO nodes;
while L contains non-PI nodes do
    take a non-PI node  $v$  from  $L$ ;
    generate a K-LUT  $v'$  to implement the function of  $v$ 
        such that  $input(v') = input(\bar{X}_v)$ ;
     $L := (L - \{v\}) \cup input(v')$ 
endwhile
end-algorithm;

```

Initialization:

- L is initialized as the list of Primary Output (PO) nodes.

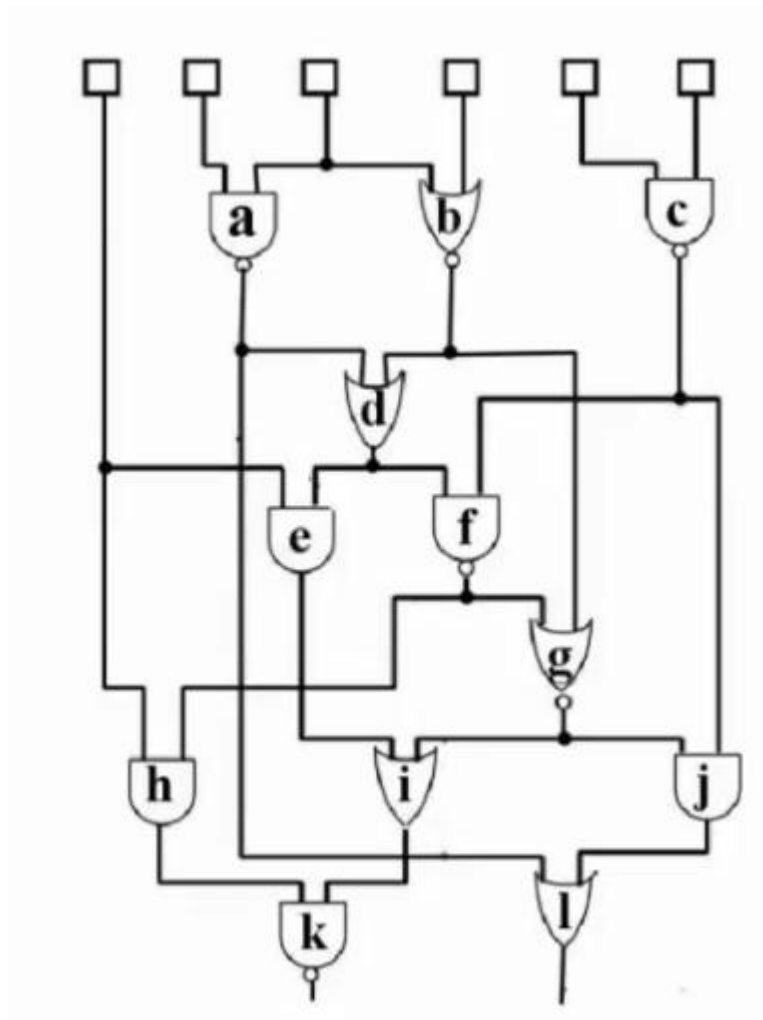
While Loop:

- The loop continues while L contains nodes that are not Primary Input (non-PI) nodes.
- Inside the loop:
 1. A non-PI node v is selected from L .
 2. A K-LUT v' is generated to implement the function of node v .
 3. The inputs to the newly generated K-LUT v' are the same as the inputs to the function X_v , representing the logic or signal of node v .
 4. After this, L is updated by removing the processed node v and adding the inputs to K-LUT v' .

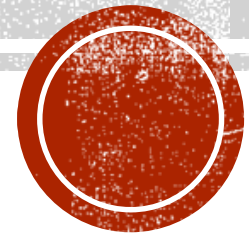
Termination:

- The loop terminates when L contains no more non-PI nodes, completing the K-LUT generation process.

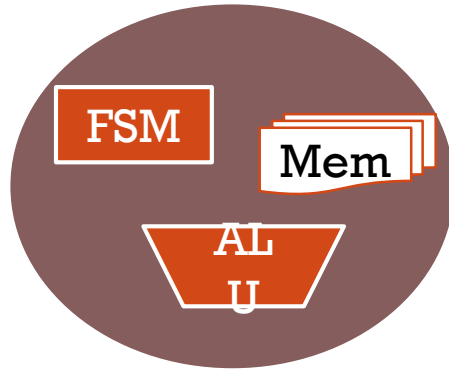
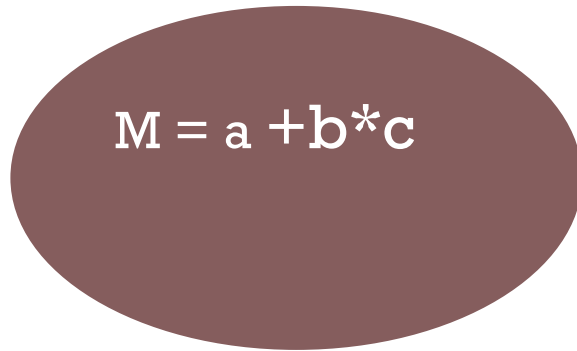




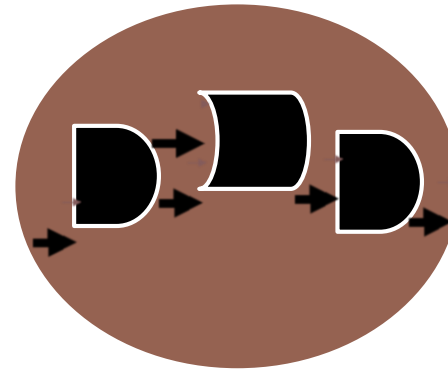
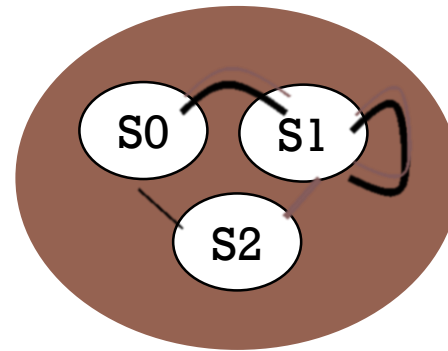
HIGH LEVEL SYNTHESIS FOR RECONFIGURABLE DEVICES



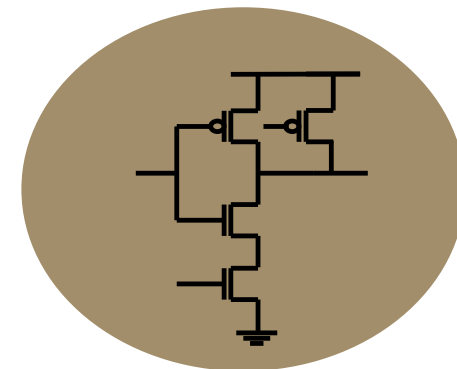
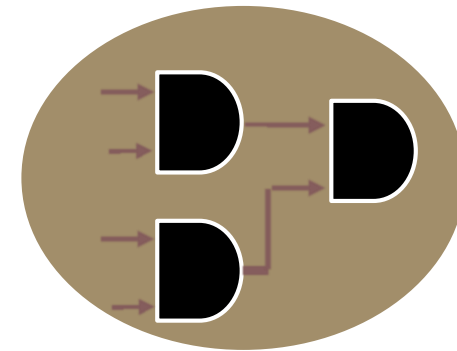
TAXONOMY OF SYNTHESIS



Architectural Synthesis



Logic Synthesis



Circuit Synthesis



Introduction

Success of Microprocessors – Ease in Programming

Mature Compilers

For three decades, a huge amount of algorithms have been developed, coded and deployed

Most of those existing programs can be executed with low modifications and low porting efforts



Sequential v/s Parallel

A degree of maturity has been reached in software programming

Parallel programming has not experienced the same advancements.

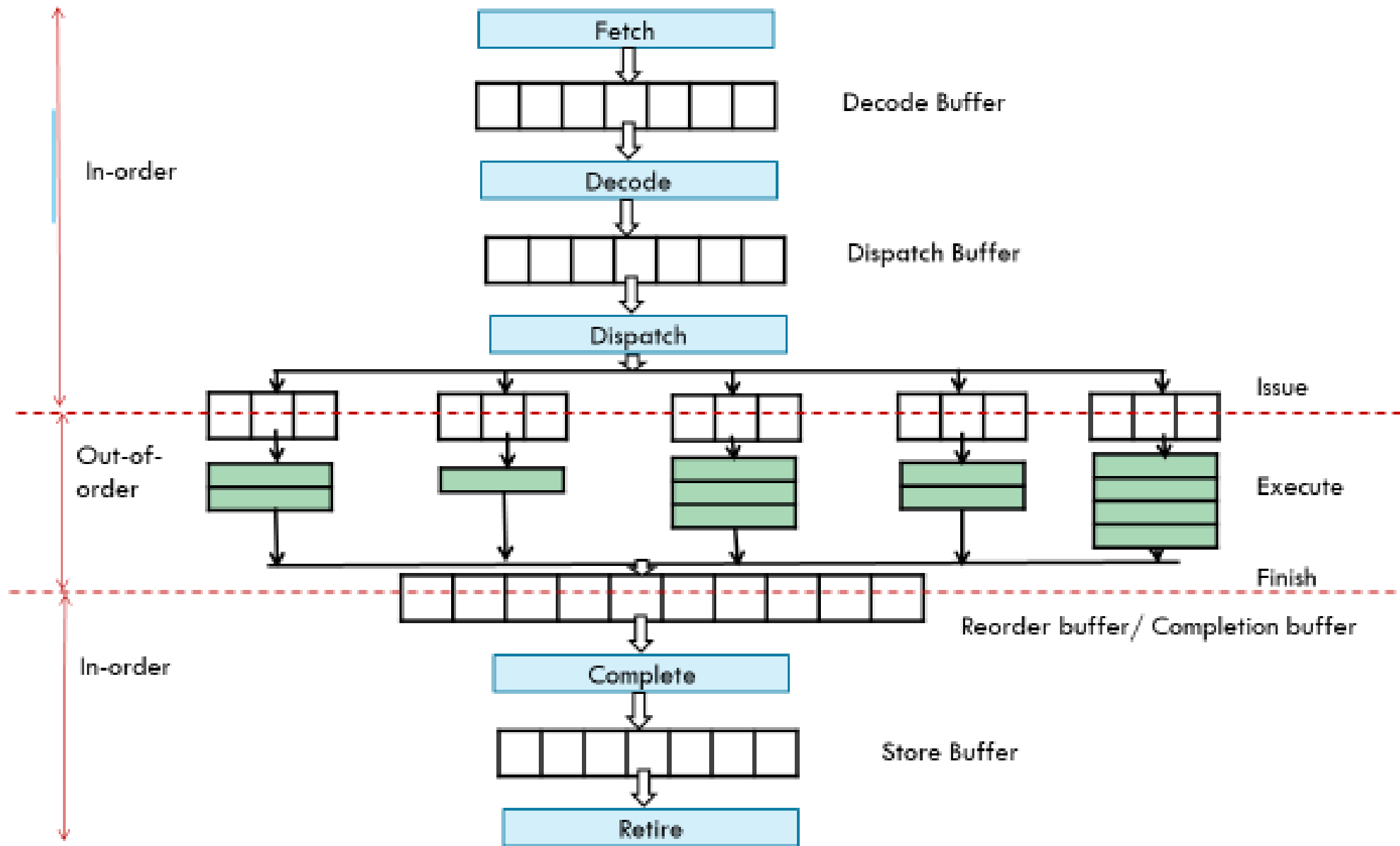
Application Partitioning

Identify Dependencies

Parallel Implementation

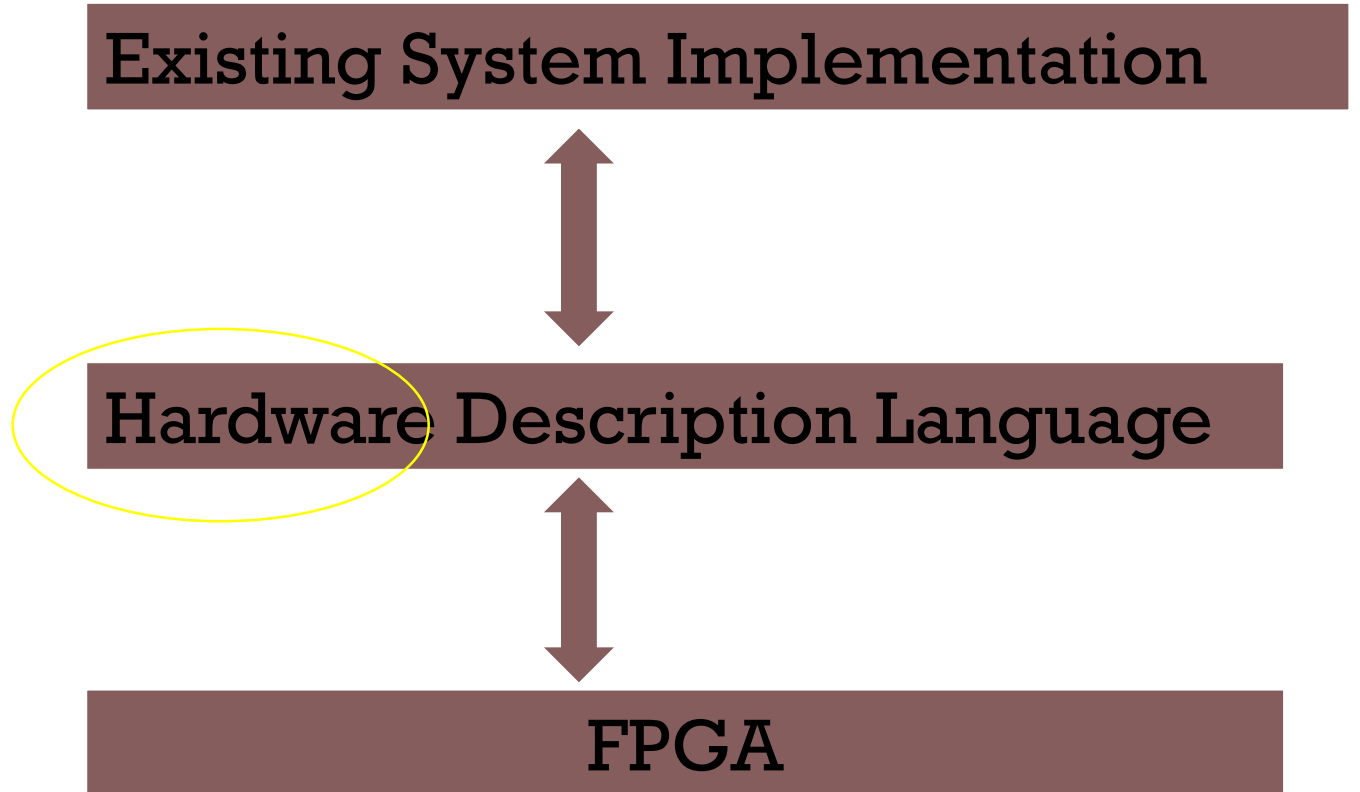
Allocate Hardware





Reconfigurable Computing

Same disadvantages as parallel computing



For reconfigurable devices, the resources on which part of the application must be mapped are not fixed.

Flexibility



High level Synthesis

Deals with specification of a given application at a very high level of abstraction as well as its implementation on a given platform.

Modelling of application can be done using FSMs

Finite State Machines (FSMs)

State charts

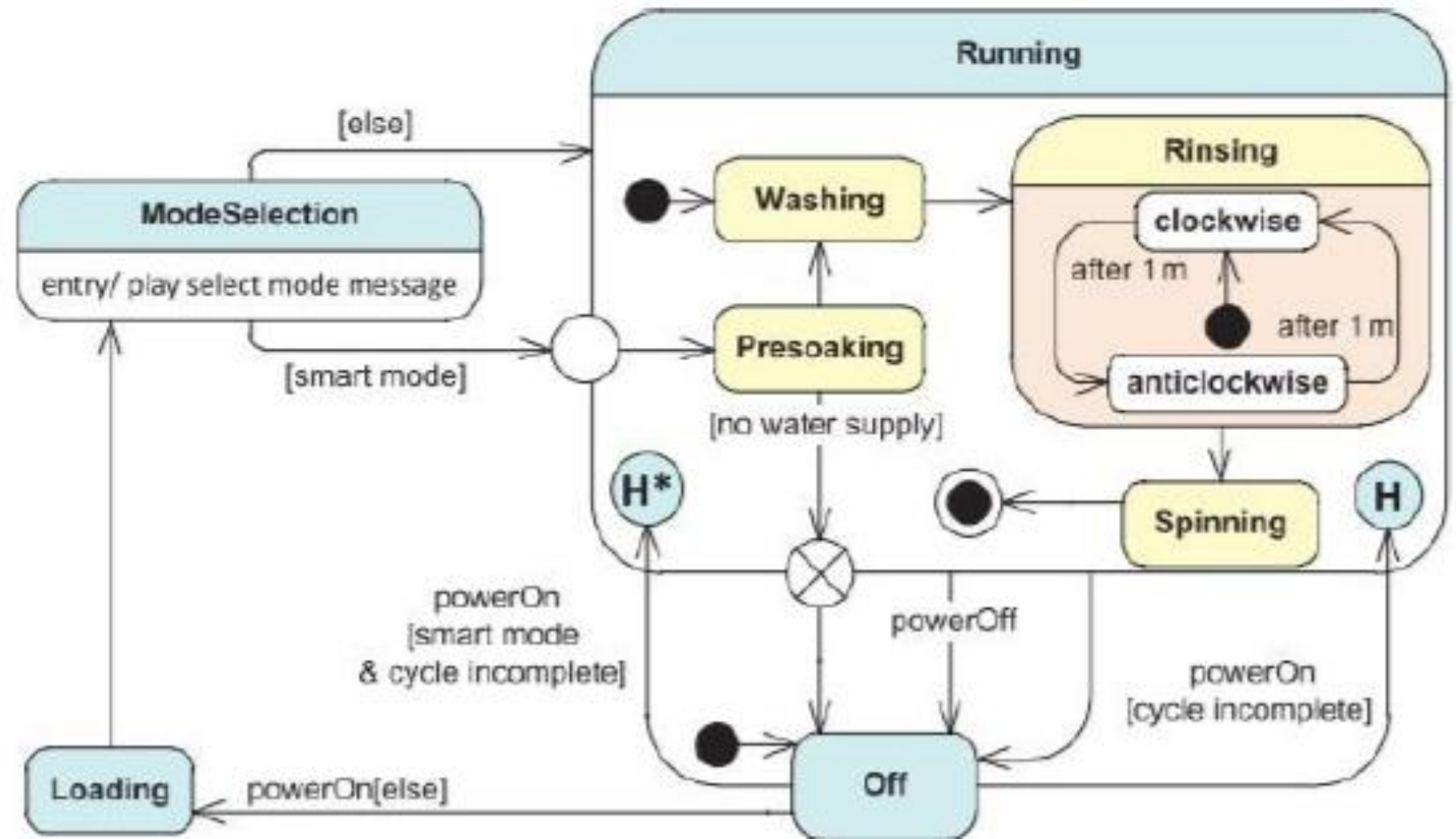
Control Dataflow Graphs

The sequencing graphs

Finite state machine with datapath



STATECHART



Dataflow Graphs

A dataflow graph (DFG) provides a means to describe a computing task in a streaming mode.

Given a code segment in a high-level language (e.g C, C++)

each operator → a node in the dataflow graph.

operand on which operator is applied → inputs of the nodes

result of the operation → output of the node

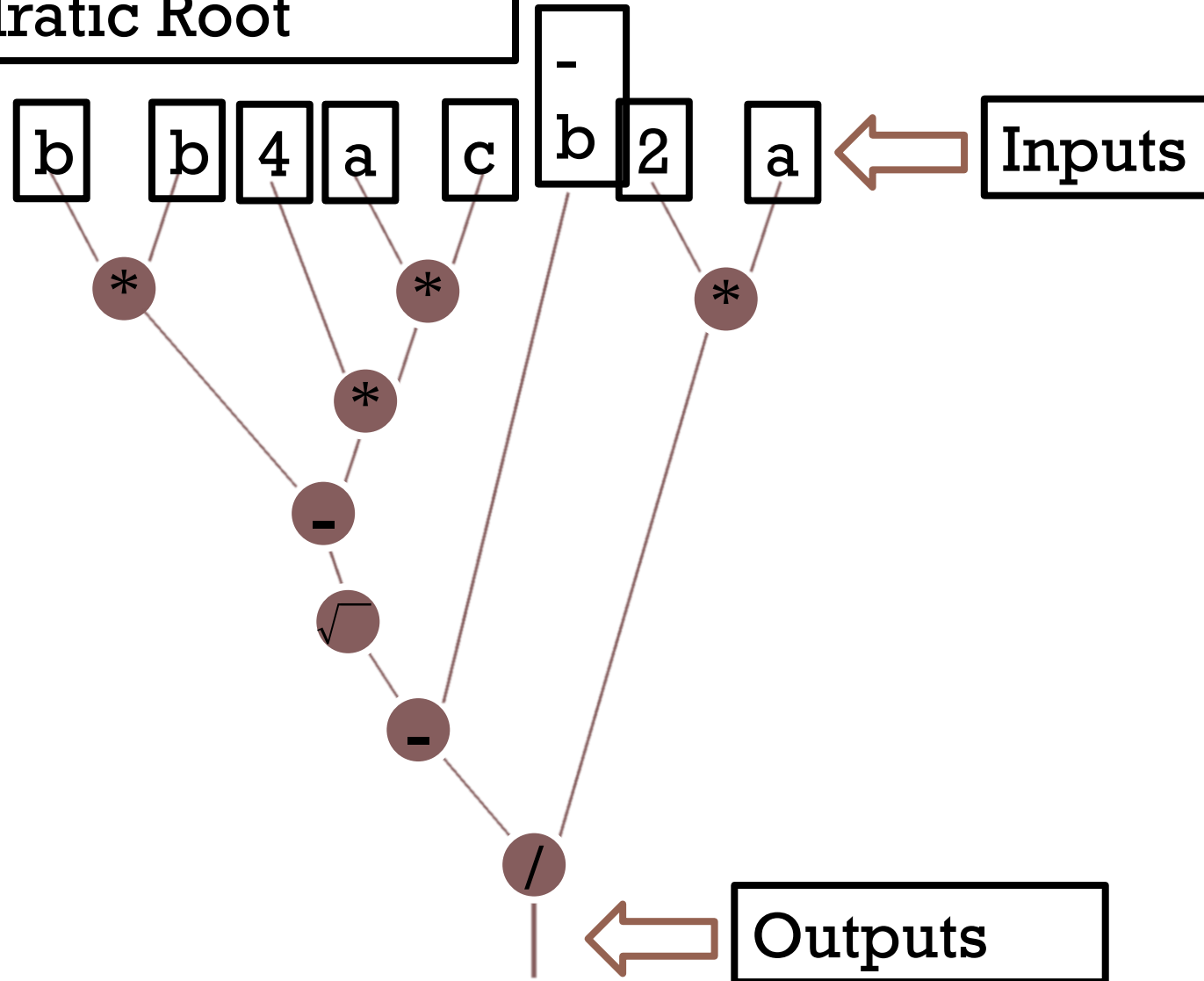
Successors and Predecessors

Data
Dependency



Dataflow Graphs

Quadratic Root



Dataflow Graphs

Given a set of tasks $T = \{T_1, \dots, T_k\}$,

A dataflow graph (DFG) is a directed acyclic graph $G = (V, E)$, where $V \rightarrow T$ is the set of nodes representing operators and E is the set of edges.

An edge $e = (v_i, v_j) \in E$ is defined through the (data) dependency between task T_i and task T_j



Dataflow Graphs

Given a node $v_i \in V$ and its implementation H_{vi} as rectangular shape module in hardware.

li denotes the length and h_i the height of H_{vi}

$ai = li \times h_i$ denotes area of H_{vi}

The latency t_i of v_i is the time it takes to compute the function of v_i using the module H_{vi}

For a given edge $e_{ij} = (v_i, v_j)$, we define the weight w_{ij} of e_{ij} as the width of bus connecting two components H_{vi} and H_{vj}

Latency t_{ij} of e_{ij} is the time needed to transmit data from H_{vi} to H_{vj}

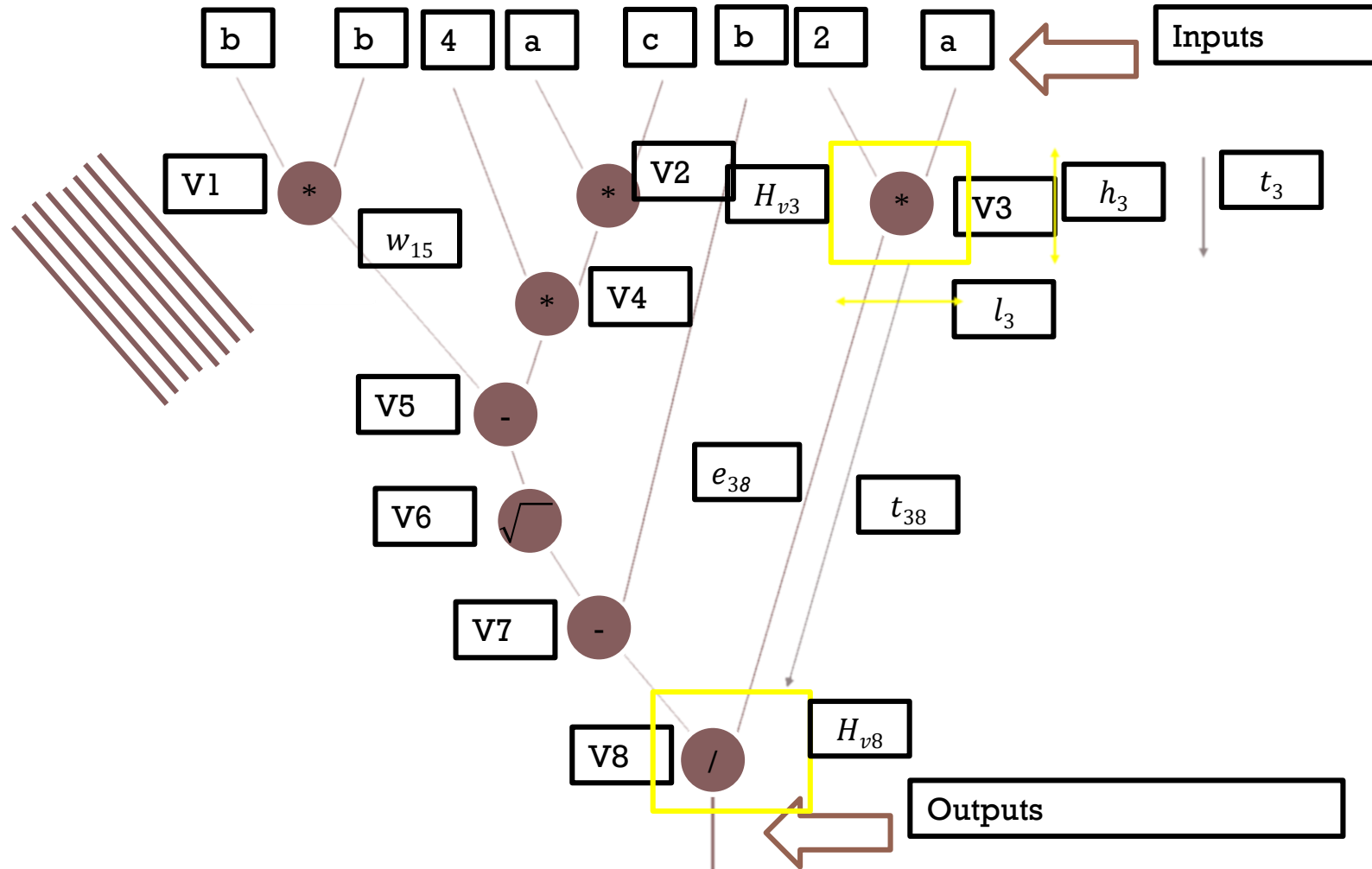


Dataflow Graphs

Quadratic Root

$$T = \{T_1 T_2 T_3 T_4 T_5 T_6 T_7 T_8\},$$

$$T = \{*,*,*,*, -, \sqrt{}, -, /\}$$



Dataflow Graphs

Any program written in a HLL can be compiled into a DFG, provided that the program is free of loops and branching instructions.

Loops and Branches are part of most programs

Several extensions of dataflow graph exist to capture program with control structures and loop.

Sequencing Graphs

Finite state machines with datapath



Sequencing Graphs

A hierarchical dataflow graph with two different types of nodes:

Operation Nodes

Normal Task Nodes in DFG

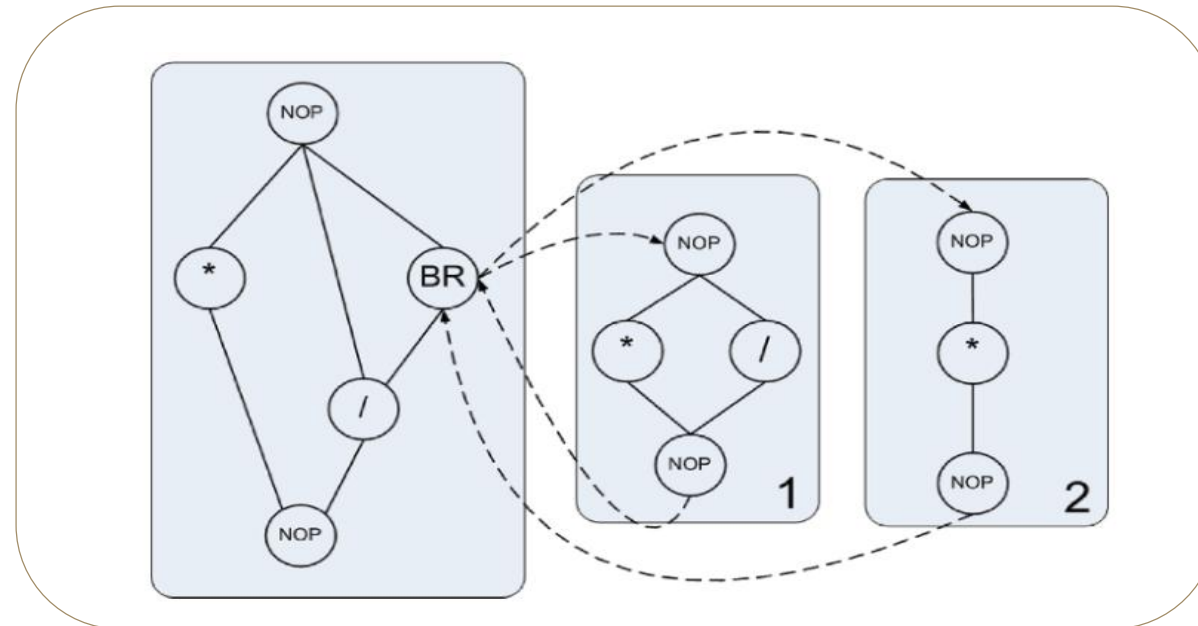
Link Nodes

Branching Nodes

Function Call

Branching

Iterations



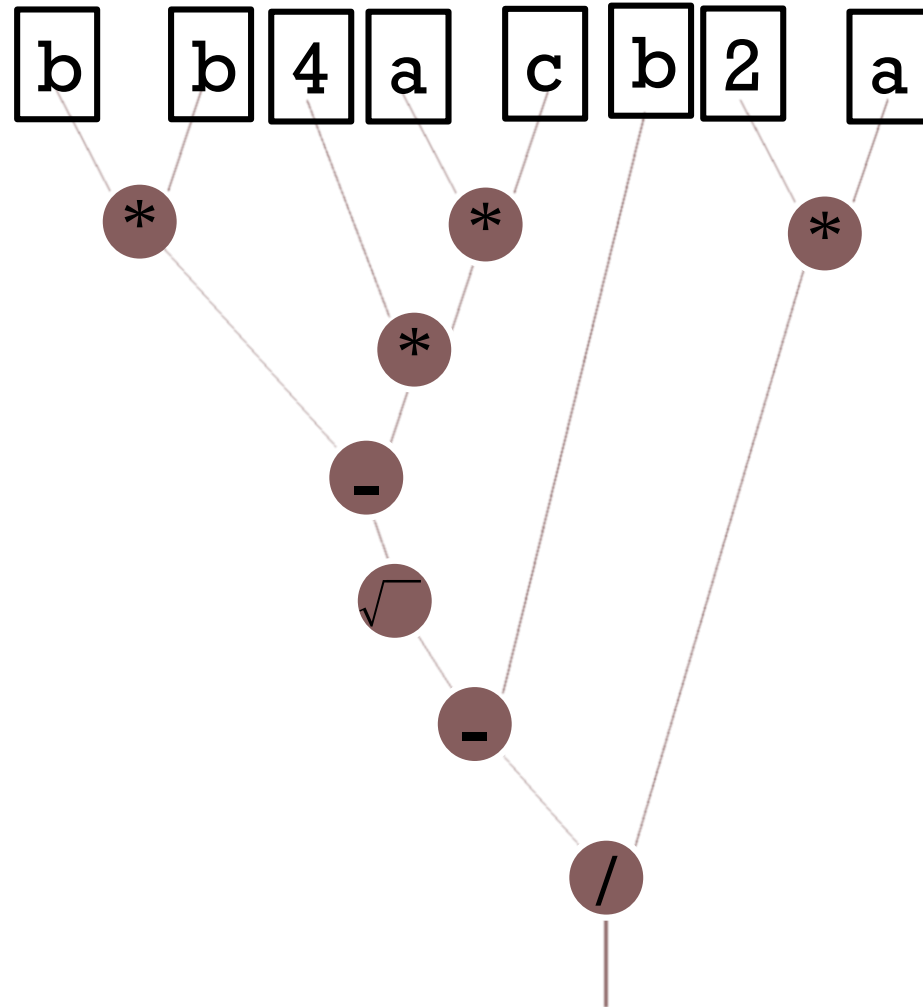
Sequencing Graphs

Source and Sink

Operation serialization

Hierarchy

Control Flow



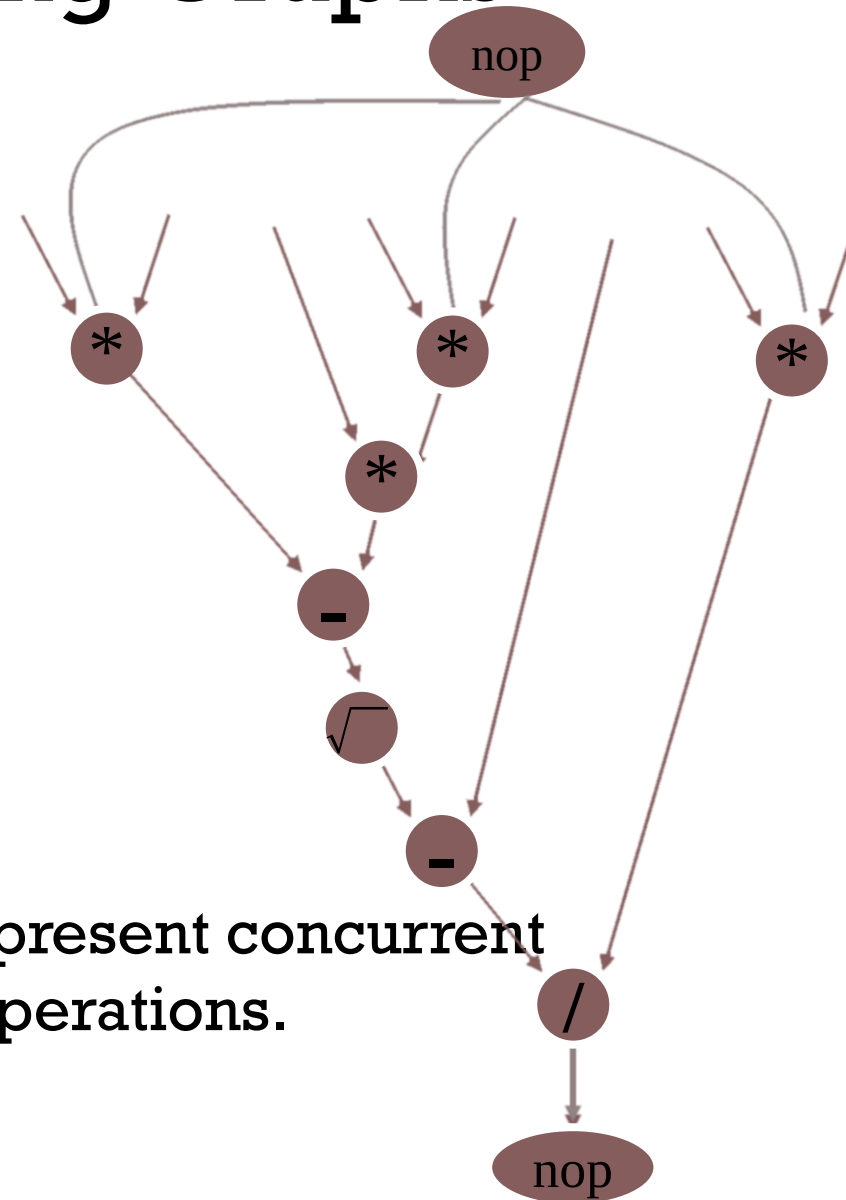
Sequencing Graphs

Source and Sink

Operation serialization

Hierarchy

Paths in the graph represent concurrent streams of operations.



Sequencing Graphs

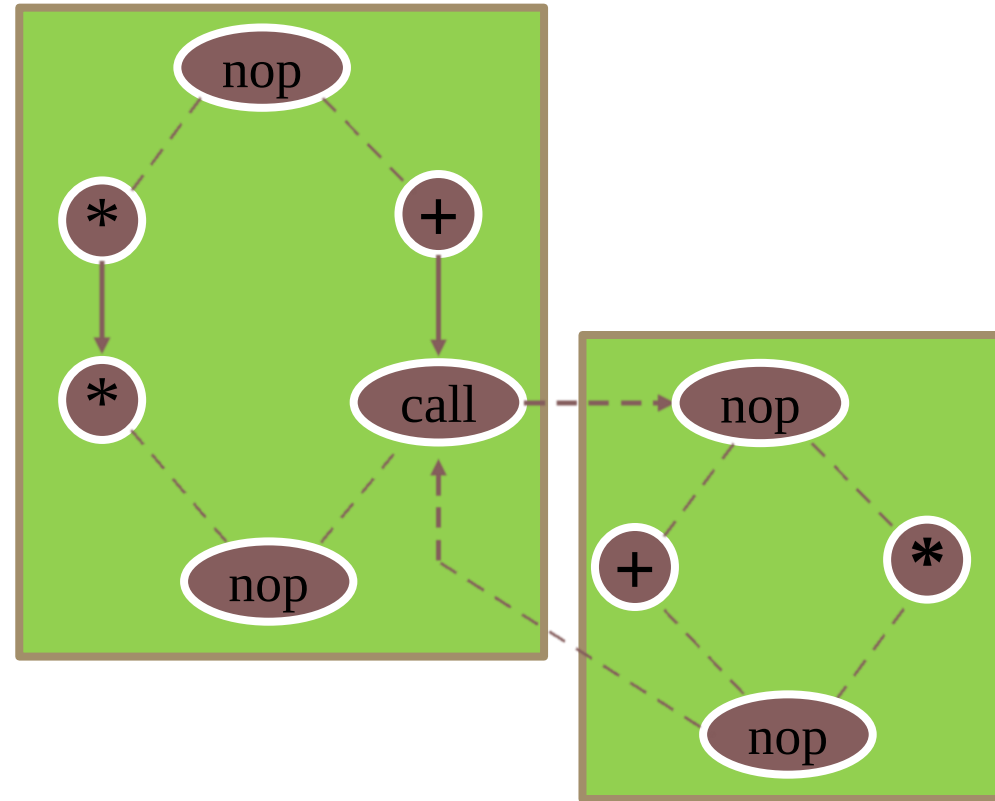
Code Sequence

$x = a * b$

$y = x * c$

$z = a + b$

$fun1(a, z)$



Where $fun1(m, n) \{p = m + n; q = m * n\}$



Sequencing Graphs

Code Sequence

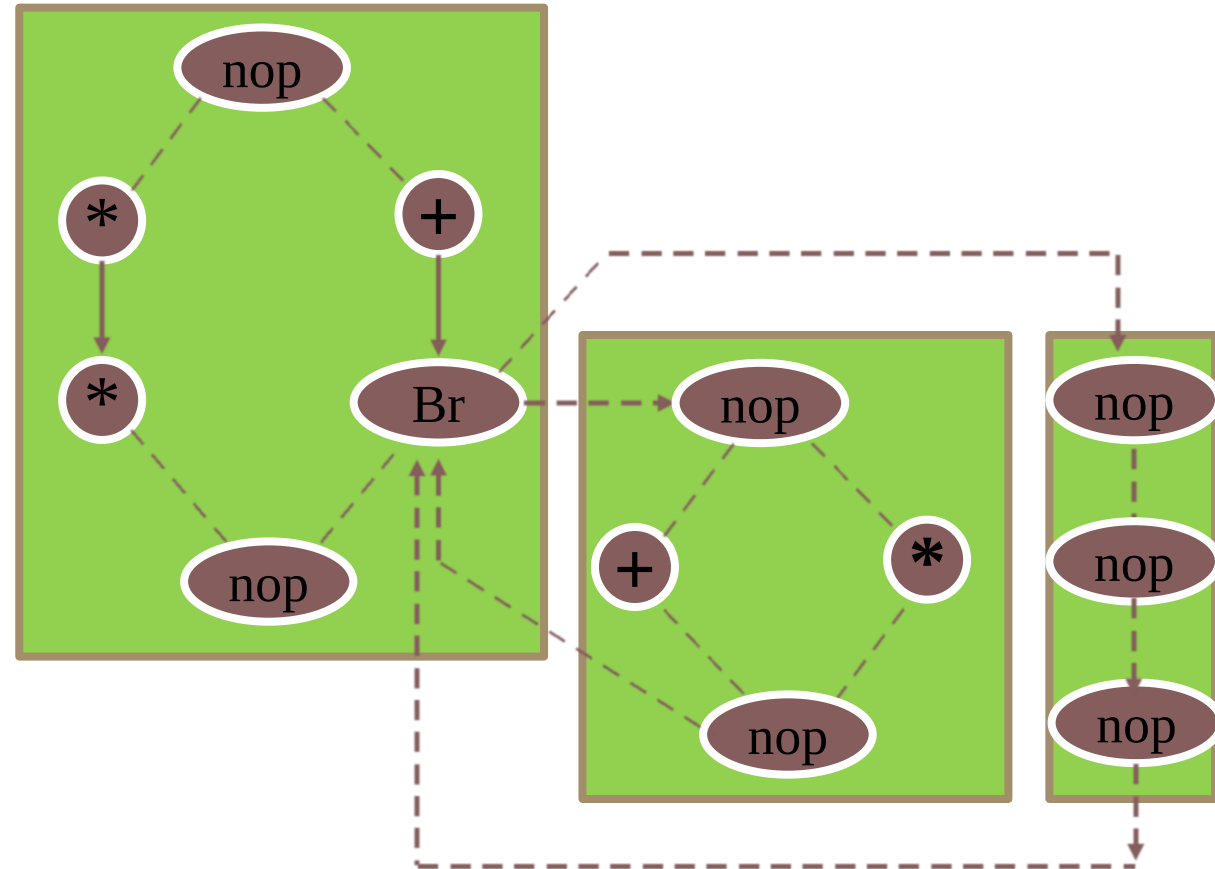
$x = a * b$

$y = x * c$

$z = a + b$

$if(z \geq 0)$

$\{p = m + n; q = m * n\}$



Sequencing Graphs

```
diffeq {  
    read(x, y, u, dx, a);  
    repeat {  
         $x_l = x + dx;$   
         $u_l = u - 3 * x * u * dx - (3 * y * dx);$   
         $y_l = y + u * dx;$   
         $c = (x < a)$   
         $x = x_l; u = u_l; y = y_l$   
    }  
    until ( c );  
    write (y);  
}
```



Sequencing Graphs

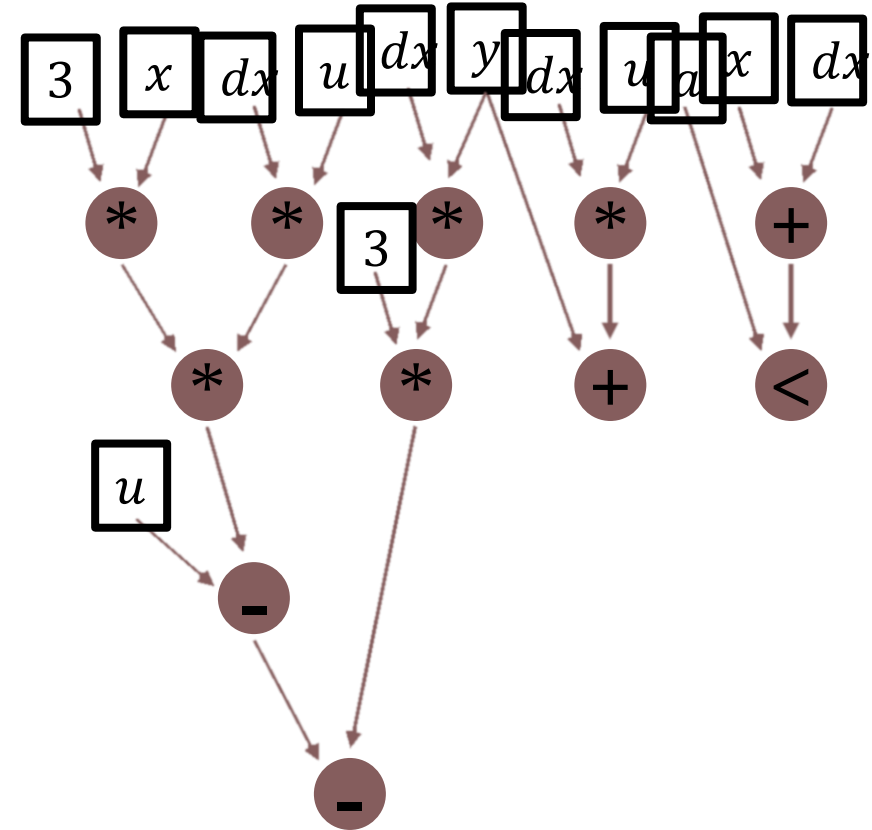
$xl = x + dx;$

$ul = u - 3 * x * u * dx - (3 * y * dx);$

$yl = y + u * dx;$

$c = (xl < a)$

$x = xl; u = ul; y = yl$



Sequencing Graphs

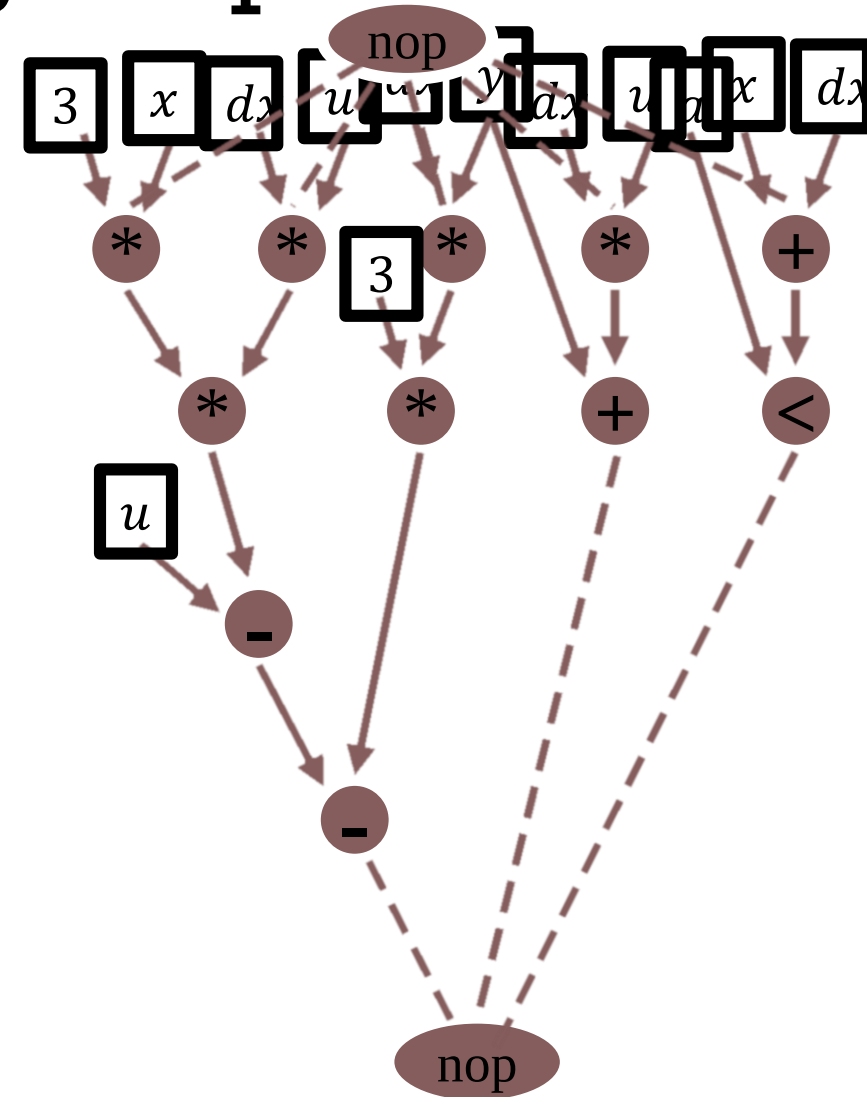
$xl = x + dx;$

$ul = u - 3 * x * u * dx - (3 * y * dx);$

$yl = y + u * dx;$

$c = (xl < a)$

$x = xl; u = ul; y = yl$

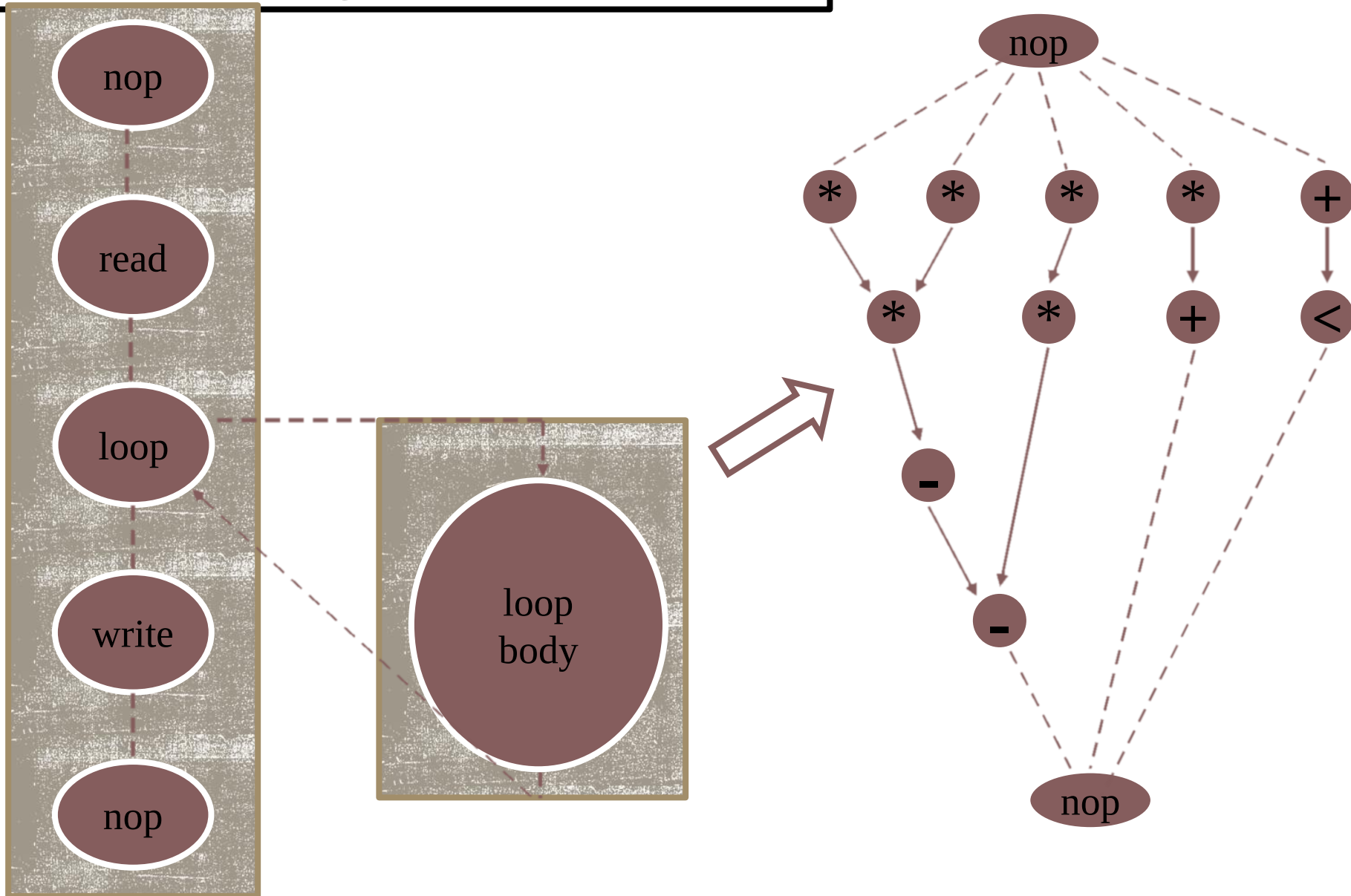


Sequencing Graphs

```
diffeq {  
    read(x, y, u, dx, a);  
    repeat {  
         $x_l = x + dx;$   
         $u_l = u - 3 * x * u * dx - (3 * y * dx);$   
         $y_l = y + u * dx;$   
         $c = (x_l < a)$   
         $x = x_l; u = u_l; y = y_l$   
    }  
    until ( c );  
    write (y);  
}
```



Sequencing Graphs



Sequencing Graphs

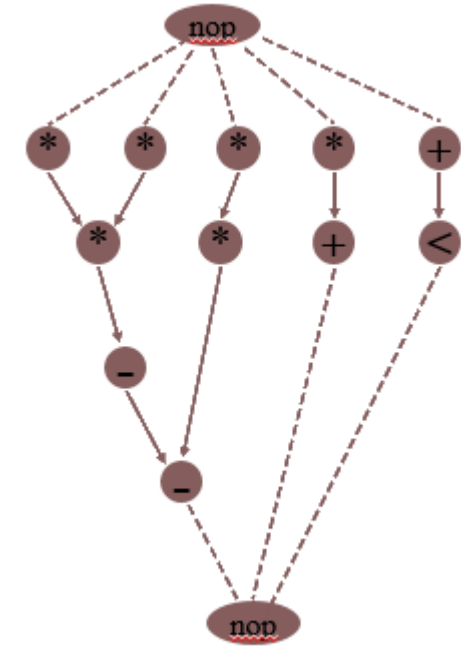
Semantics of sequencing graphs

Marking of vertices

Waiting for execution.

Executing.

Have completed execution.



Firing an operation means starting its execution.

Execution semantics

An operation can be fired as soon as all its immediate predecessors have completed execution.



Sequencing Graphs

Semantics of sequencing graphs

Model can be reset by making all operations waiting for execution.

Model can be fired (executed) by firing the source vertex.

Model completes execution when sink completes execution.

